

Predicting Defects at the Method Level

A Machine Learning & "What-If" Analysis of
Apache Bookkeeper & Apache OpenJPA

Analysis by Flavio Simonelli

Ingegneria del Software 2

Indice

1. Introduzione

- Il problema della manutenibilità
- Le domande di ricerca

2. Metodologia

- Creazione dei Dataset
- Addestramento e Inferenza

3. Risultati e Analisi

- Confronto dei modelli predittivi
- Refactoring guidato dai dati
- Analisi "What-If" no smells

4. RoadMap

Debito Tecnico e Granularità

Analisi a Livello di Classe (Approccio Tradizionale)

- Eccessiva Rumorosità: interi moduli segnalati come difettosi, rendono difficile l'isolamento delle cause.
- Bassa Azionabilità: difficoltà nell'identificare il punto esatto dove intervenire con il refactoring.

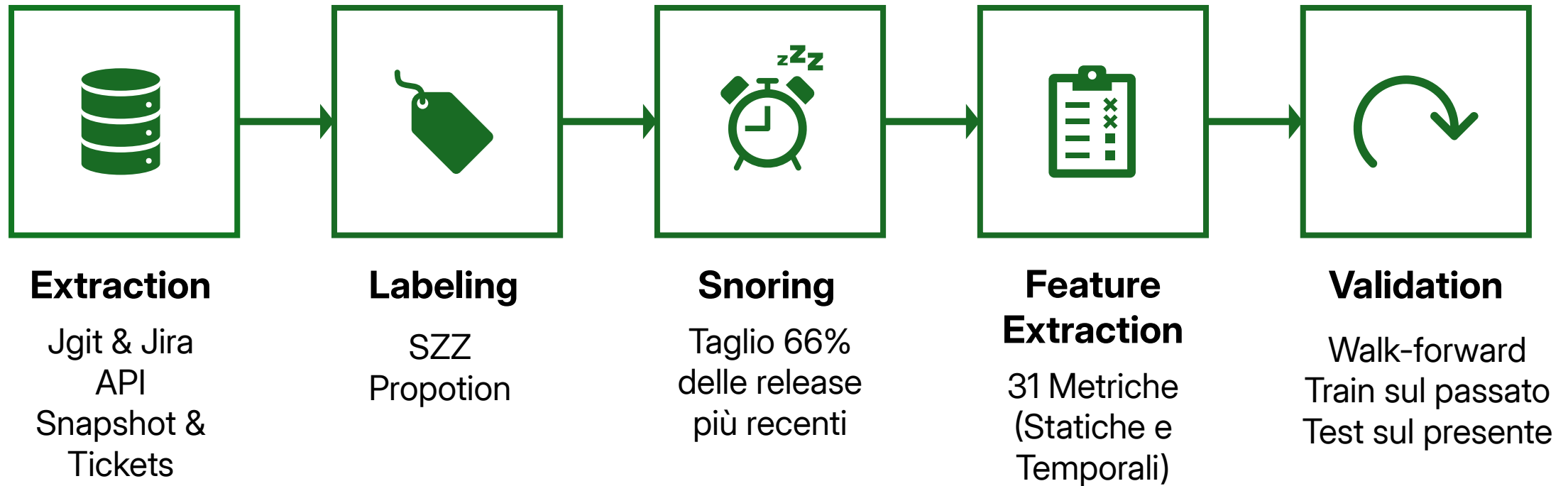
Analisi a Livello di Metodo (Approccio Proposto)

- Granularità Mirata: Identificazione chirurgica delle porzioni di codice prone ai bug.
- Ottimizzazione del ROI (Return on Investment): Focalizzazione degli sforzi di manutenzione solo sulle funzioni a più alto rischio.

Le domande di ricerca

- **RQ1 (Valutazione dei Predittori):**
Quale classificatore presenta il grado di accuratezza superiore tra quelli analizzati?
- **RQ2 (Efficacia del Refactoring):**
In che misura l'intervento di refactoring sul metodo più critico (scelto in base alla metrica più correlata al bug nell'ultima release) ha influenzato le features correlate alla bugginess?
- **RQ3 (Analisi What-If):**
Quanti metodi difettosi sarebbero potuti essere evitati eliminando completamente gli smell di codice (scenario zero-smells)?

ML pipeline



Creazione del Dataset

- **Progetti analizzati:** Apache Bookkeeper e OpenJPA
- **Selezione delle versioni**  **Jira**
 - Le release ufficiali registrate su Jira fungono da punto di riferimento temporale primario
 - Sono state eliminate le versioni indicate come non rilasciate o prive di data di rilascio
- **Estrazione dal Version Control**  **git**
 - Per ogni release Jira, lo snapshot del sistema è stato ricostruito posizionandosi sull'ultimo commit effettuato entro la data di rilascio

Labeling e Propotion

- **Problema:** Molti ticket Jira riportano la Fixed Version (FV) ma mancano della Injected Version (IV).
- **Soluzione:** Applicazione dell'euristica SZZ Propotion
- **P (propotion factor):** Calcolato con un approccio Moving Window (dimensione 1% Ticket con IV nota) basato sui ticket passati con IV nota.
- **Fix-commit:** Identification dei commit di correzione tramite la ricerca dell'ID del ticket nei messaggi di commit.
- **Labeling del Metodo:** Un metodo è etichettato come BUGGY in tutte le versioni comprese nell'intervallo [IV,FV).

Apache OpenJPA

Ticket	Conteggio	Percentuale
Totale Utilizzati / Processati	825	
<i>Dettaglio sui ticket utilizzati:</i>		
Con Affected Version	704	85,33%
Con Fix Version	801	97,09%
Con entrambe (AV + FV)	693	84,00%

Formule propotion

$$IV = FV - (P \cdot (FV - OV))$$

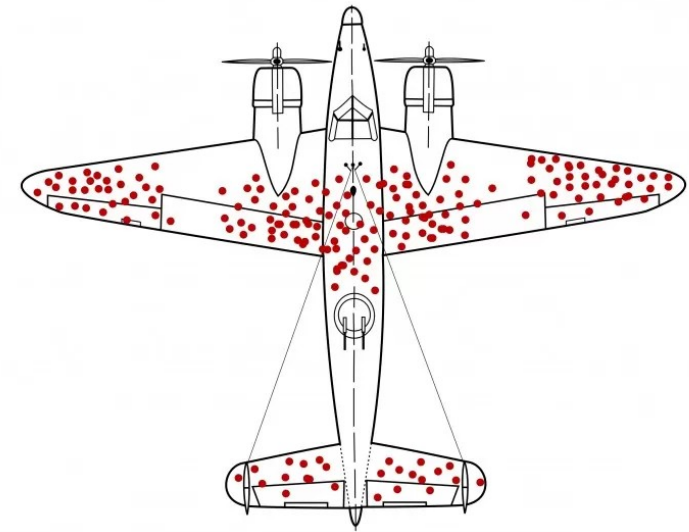
$$P = \frac{FV - IV}{FV - OV}$$

Apache Bookkeeper

Ticket	Conteggio	Percentuale
Totale Utilizzati / Processati	286	
<i>Dettaglio sui ticket utilizzati:</i>		
Con Affected Version	133	46,50%
Con Fix Version	283	98,95%
Con entrambe (AV + FV)	133	46,50%

Snoring

- **Taglio del Dataset:** Per garantire l'integrità dei dati, è stato effettuato un taglio netto delle release finali.
- **Percentuale di scarto:** È stato rimosso l'ultimo 66% delle release per entrambi i progetti (Bookkeeper 5 su 14 e OpenJPA 14 su 42).
- **Obiettivo:** Assicurare un intervallo di osservazione sufficientemente lungo affinché i bug latenti possano emergere ed essere etichettati correttamente.



Il Survivorship Bias come metafora del Code Snoring. Le zone senza 'colpi' (bug) nelle release recenti sono spesso difettosità latenti non ancora rilevate.

Le metriche raccolte

Metrica	Tipo	Scope		Aggregazioni			Descrizione
		Loc	Glob	Sum	Max	Avg	
LOC	Statica	-	-	-	-	-	Linee di codice fisiche.
StatementCount	Statica	-	-	-	-	-	Istruzioni eseguibili.
ParameterCount	Statica	-	-	-	-	-	Numero di parametri.
LocalVarCount	Statica	-	-	-	-	-	Variabili locali dichiarate.
Cyclomatic	Statica	-	-	-	-	-	Complessità di McCabe.
Cognitive	Statica	-	-	-	-	-	Complessità Cognitiva.
NestingDepth	Statica	-	-	-	-	-	Profondità annidamento.
ReturnComp	Statica	-	-	-	-	-	Complessità tipo di ritorno.
NSmells	Statica	-	-	-	-	-	Violazioni PMD.
NR	Processo	✓	✓	✓	-	-	Numero di revisioni (commit).
NAuth	Processo	✓	✓	✓	-	-	Numero autori unici.
LOC_Added	Processo	✓	✓	✓	✓	✓	Linee di codice aggiunte.
LOC_Deleted	Processo	✓	✓	✓	✓	✓	Linee di codice rimosse.
Churn	Processo	✓	✓	✓	✓	✓	Turbolenza (Added + Deleted).

Dimensioni temporali: **Loc** (Locale): Commit tra release corrente e precedente | **Glob** (Globale): Storico dall'inserimento del metodo

Operatori di Aggregazione: **Sum**: Somma | **Max**: Valore Massimo | **Avg**: Media

Data Balancing

- **Problema:** La natura dei Dataset software è intrinsecamente sbilanciata (poche istanze 'Buggy' rispetto a 'Clean').
- **Tecniche Applicate:**
 - SMOTE (Synthetic Minority Over-Sampling): Generazione di istanze sintetiche della classe minoritaria per il progetto Bookkeeper.
 - Under-sampling: Riduzione casuale della classe maggioritaria per il progetto OpenJPA.
- **Obiettivo:** Prevenire il bias del classificatore verso la classe maggioritaria e massimizzare la Recall.

Apache Bookkeeper

Totale righe scritte	21432
Totale righe BUGGY	6790
Percentuale di bugginess nel dataset	31,68%

Dataset originale senza applicazione di sampling

Apache OpenJPA

Totale righe scritte	244608
Totale righe BUGGY	53604
Percentuale di bugginess nel dataset	21,91%

Dataset originale senza applicazione di sampling

Feature Selection

- **Global Feature Selection**

- La selezione delle metriche viene eseguita una sola volta sull'intero dataset prima della validazione.
- Introduce un "bias di selezione" perché l'algoritmo vede informazioni provenienti dai test futuri durante la scelta delle feature.

- **Walk-forward / Per-Fold Selection**

- La selezione viene ricalcolata dinamicamente ad ogni fold.
- Utilizza esclusivamente i dati di training disponibili ad ogni iterazione.
- Si adatta all'evoluzione del software, catturando come la rilevanza delle metriche cambi nel tempo.

- **Tecniche applicate:**

- Ranking Filter (InfoGain): Valutazione del guadagno informativo per ogni singola metrica.
- Subset Filter (BestFirst + CFS): Ricerca di sottoinsieme di feature che massimizzano la correlazione con i bug minimizzando la ridondanza tramite un approccio forward-search.

Validazione Walk-Forward

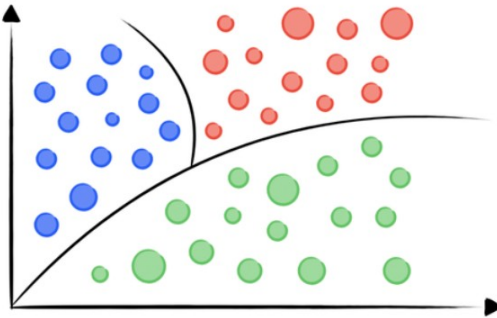
- **Protocollo Temporale:** il dataset non viene diviso casualmente, ma segue rigorosamente l'ordine cronologico delle release.
- **Addestramento sequenziale:** il modello viene addestrato sulle release passate per testare le performance esclusivamente sulla release corrente.
- **Prevenzione del Data Leakage:** evita di utilizzare informazioni future per predire il passato, garantendo l'integrità scientifica dei risultati.
- **Iterazioni:** il processo è stato eseguito 10 volte, come previsto dalla letteratura, per assicurare la stabilità delle metriche.

Run	Part				
	1	2	3	4	5
1	Testing	Training	Training	Training	Training
2	Training	Testing	Training	Training	Training
3	Training	Training	Testing	Training	Training
4	Training	Training	Training	Testing	Training
5	Training	Training	Training	Training	Testing

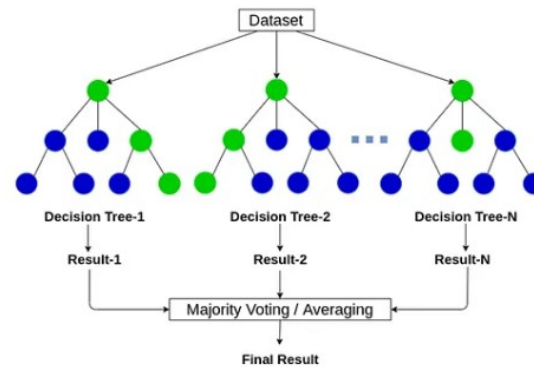
Testing
Training

Modelli considerati

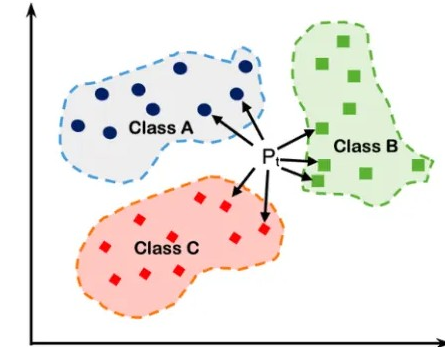
Naive Bayes



Random Forest



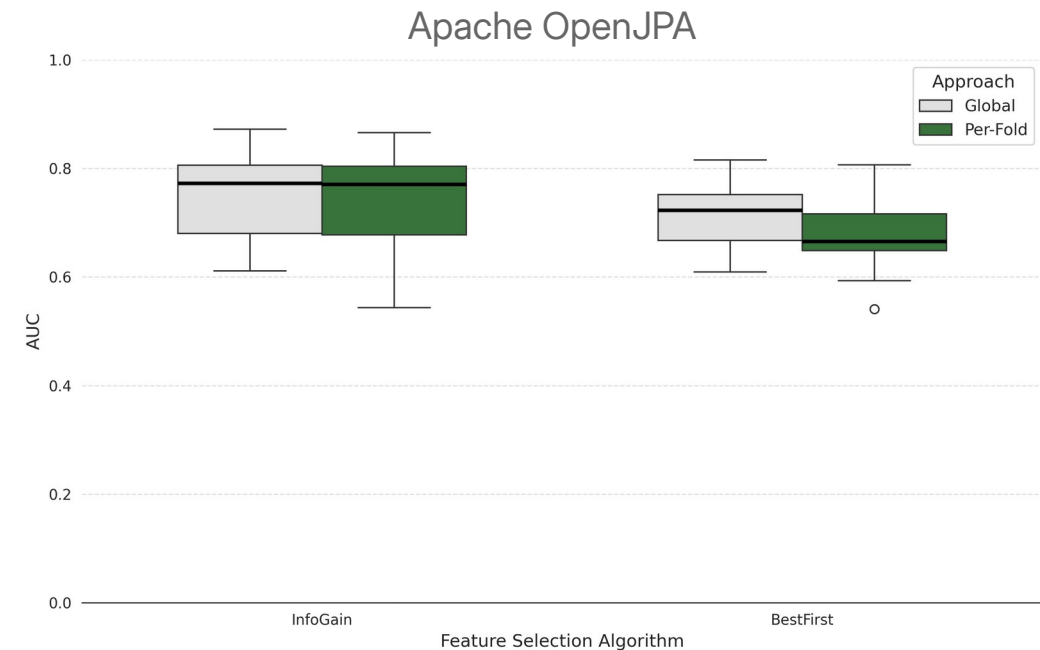
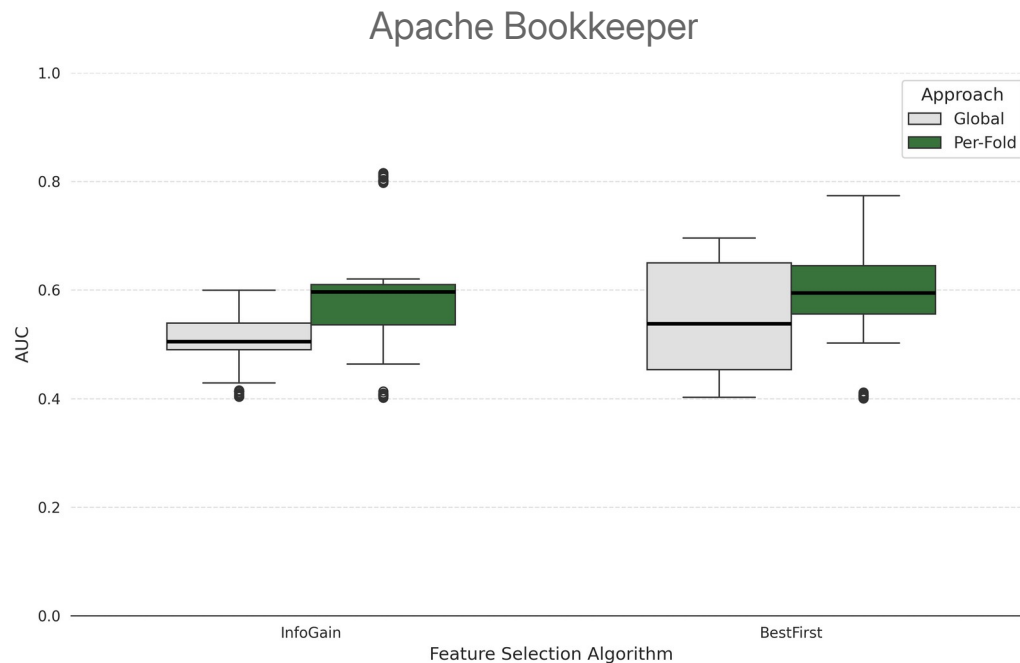
IBk (K-NN)



RQ1: Approccio Per-Fold vs Global

Bookkeeper: Vantaggio della selezione locale.

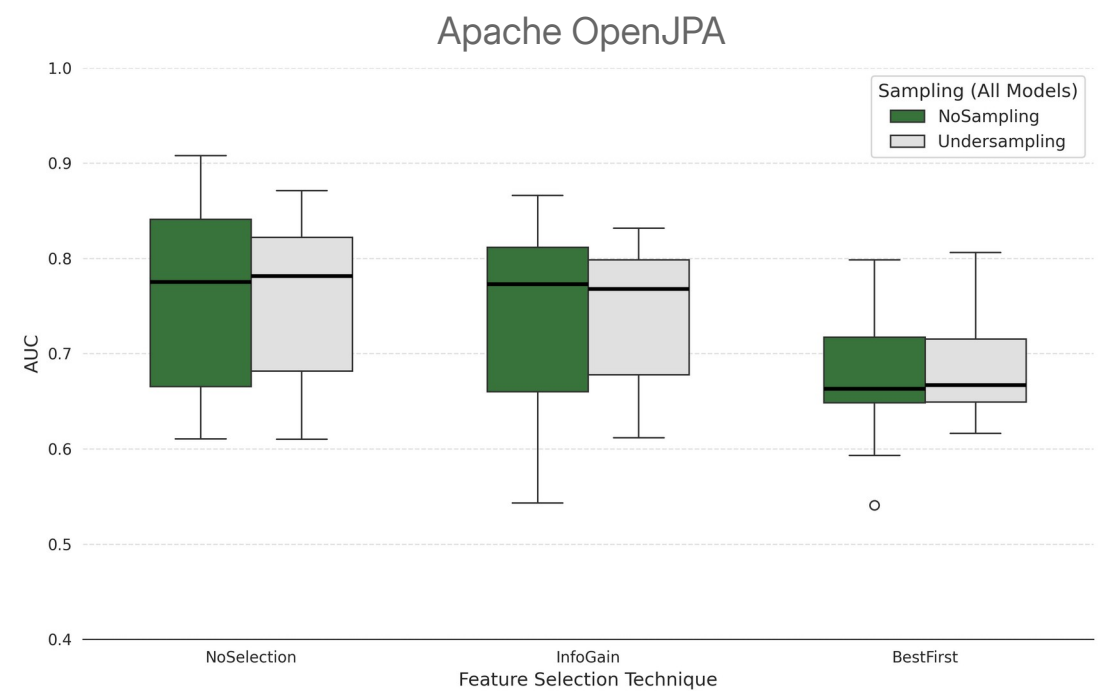
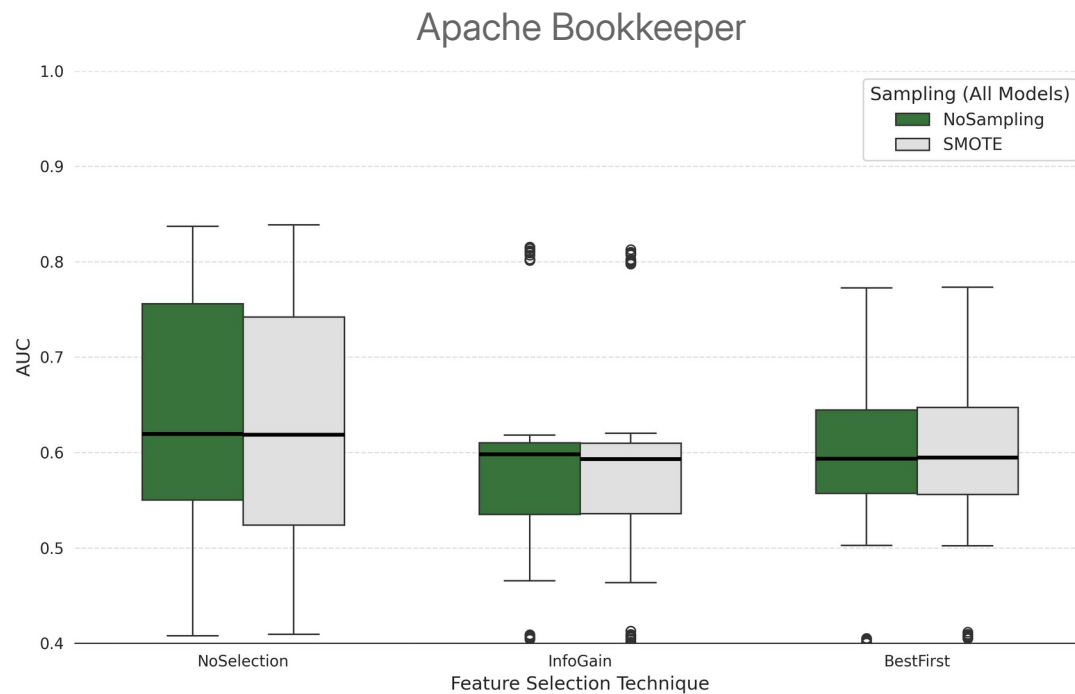
OpenJPA: Impatto del bias di selezione teorico.



Le distribuzioni riportano l'AUC aggregata su tutti i modelli (3) e le strategie di campionamento (2).

RQ1: Feature Selection e Sampling a confronto

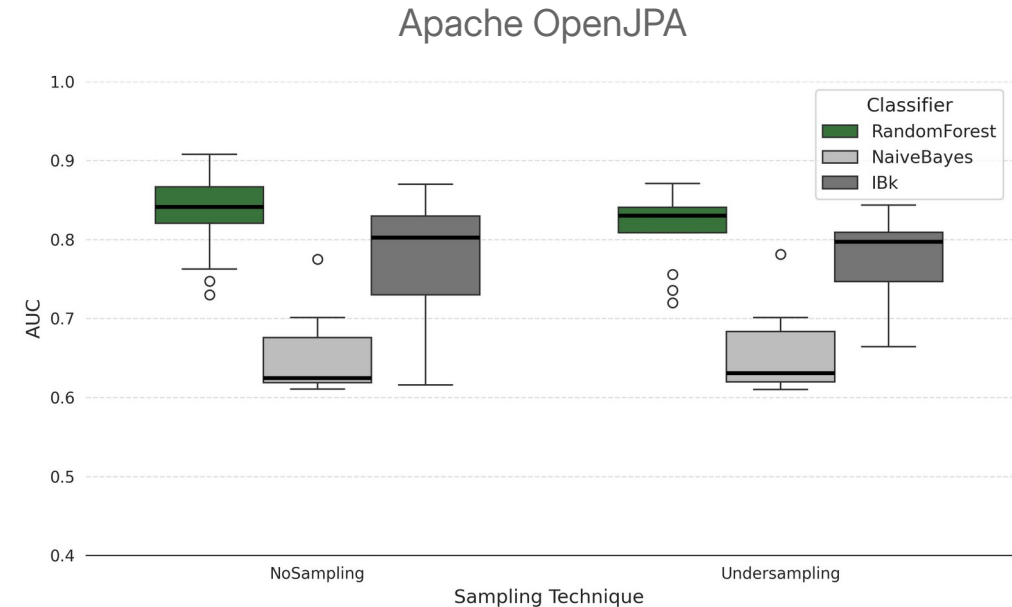
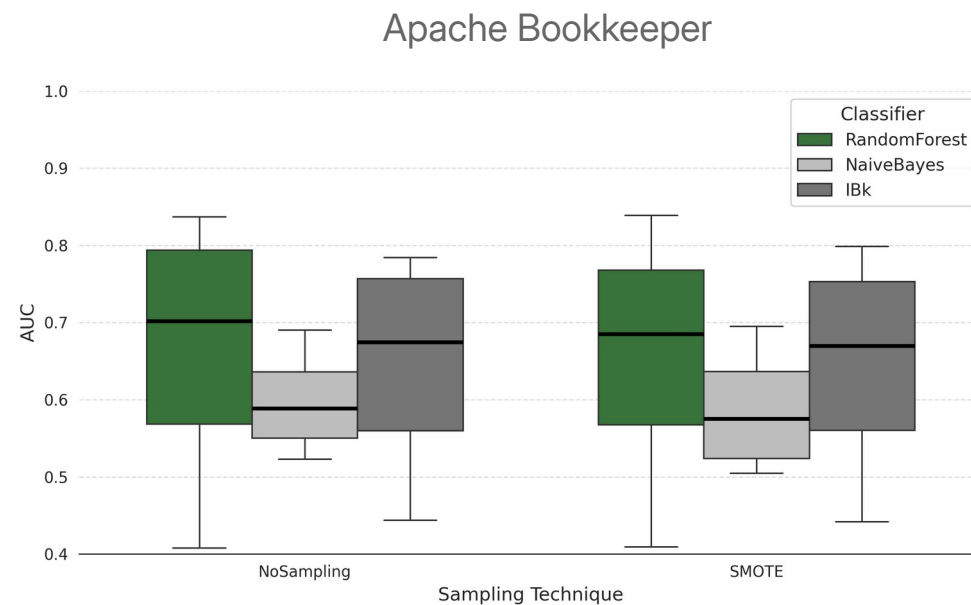
No Selection: L'integrità del dataset originale ha dimostrato performance superiori e una minore varianza nelle predizioni.



Distribuzioni AUC aggregate su tutti i modelli.

RQ1: Scelta del Modello

Il modello **Random Forest (No Selection / No Sampling)** si conferma il più efficace e stabile per entrambi i casi di studio.



Distribuzioni AUC per NoSelection, separate per classificatore.

RQ2: Correlazione feature vs Bugginess

In entrambi i casi studio, la **LOC** (Lines of Code) si conferma la metrica azionabile con la più alta correlazione alla bugginess.

Apache Bookkeeper

Metrica	Pearson (r)	Spearman (ρ)
NR	0.1666	0.2816
NAuth	0.2638	0.2802
MAX_Churn	0.0313	0.2774
Churn	0.0367	0.2774
AVG_Churn	0.0283	0.2773
AVG_LOC_Added	0.0210	0.2729
MAX_LOC_Added	0.0237	0.2728
LOC_Added	0.0270	0.2728
MAX_LOC_Deleted	0.0671	0.2673
AVG_LOC_Deleted	0.0658	0.2671
LOC_Deleted	0.0778	0.2666
Global_LOC_Deleted	0.0568	0.1191
Global_MAX_LOC_Deleted	0.0453	0.1180
Global_AVG_LOC_Deleted	0.0483	0.1140
LOC	0.1147	0.1101
NumStatements	0.0839	0.0996
Cyclomatic	0.0999	0.0969
CognitiveComplexity	0.0908	0.0943
NestingDepth	0.0858	0.0912
CodeSmells	0.0874	0.0876
Global_NR	0.1011	0.0810
Global_NAuth	0.0874	0.0759
Global_Churn	0.0105	0.0674
Global_MAX_Churn	0.0048	0.0630
Global_AVG_Churn	-0.0082	0.0588
Global_LOC_Added	0.0027	0.0578
Global_MAX_LOC_Added	-0.0007	0.0549
Global_AVG_LOC_Added	-0.0159	0.0508
NumLocalVars	0.0572	0.0472
ReturnTypeComplexity	-0.0223	-0.0295
NumParams	-0.0178	-0.0227

Apache OpenJPA

Metrica	Pea.	Spear.
LOC	0.219	0.234
NumLocalVars	0.198	0.226
NumStatements	0.200	0.220
NestingDepth	0.206	0.210
Cyclomatic	0.173	0.204
CognitiveCplx	0.195	0.201
Global_NAuth	0.239	0.193
Global_NR	0.190	0.188
Glob_LOC_Add	0.022	0.181
Glob_MAX_Add	0.014	0.174
Glob_AVG_Add	0.003	0.171
Glob_Churn	0.027	0.169
Glob_LOC_Del	0.034	0.163
Glob_MAX_Churn	0.019	0.161
Glob_MAX_Del	0.025	0.155
Glob_AVG_Churn	0.009	0.153
Glob_AVG_Del	0.021	0.145
AVG_LOC_Added	0.013	0.134
LOC_Added	0.008	0.134
MAX_LOC_Added	0.014	0.134
NAuth	0.138	0.132
NR	0.097	0.132
Churn	0.012	0.132
MAX_Churn	0.016	0.132
AVG_Churn	0.017	0.131
CodeSmells	0.144	0.115
LOC_Deleted	0.018	0.115
MAX_LOC_Del	0.017	0.115
AVG_LOC_Del	0.021	0.115
NumParams	0.043	0.040
ReturnCplx	0.036	0.036

RQ2: Efficacia del Refactoring Bookkeeper

- **Intervento:** Decomposizione tramite pattern Extract Method (144 LOC → 5 metodi).
- **Risultato:** Il modello ha invertito la predizione da BUGGY a CLEAN per tutti i nuovi metodi.
- **Driver del cambiamento:** L'abbattimento drastico della complessità ciclomatica (da 16 a 3) e dei Code Smells (da 17 a 3) insieme alla LOC hanno neutralizzato l'effetto negativo del Churn elevato dovuto alla ristrutturazione.

Tabella sintetica con alcune delle metriche principali

Metrica	Pre-Refactoring	Post-Refactoring (M3)	Trend
LOC	144	39	↓
Statements	73	25	↓
Params	1	1	=
Vars	37	13	↓
Cyclomatic Complexity	16	3	↓
Cognitive Complexity	17	2	↓
Nesting Depth	3	1	↓
Return Type Complexity	1	1	=
Code Smells (PMD)	17	3	↓
Deleted	3	108	↑
Churn	6	111	↑
Predizione Modello	BUGGY (TRUE)	CLEAN (LO Risk)	

```
org.apache.bookkeeper.benchmark.BenchThroughpu  
tLatency.main(String[])
```

RQ2: Efficacia del Refactoring OpenJPA

- **Intervento:** Refactoring tramite pattern Extract Method (185 LOC → 15 metodi).
- **Risultato:** Compartimentazione del problema. 14 metodi su 15 sono diventati CLEAN, ma il metodo core è rimasto classificato come BUGGY.
- **Analisi del limite:** Nonostante la pulizia del codice (LOC e Complessità ridotte), la turbolenza storica (Global Churn) e il volume di codice cancellato (Deleted) hanno mantenuto alta l'allerta del modello, dimostrando che il rischio non dipende solo dalla forma attuale del codice ma anche dalla sua storia.

Tabella sintetica con alcune delle metriche principali

Metrica	Pre-Refactoring	Post-Refactoring (M5)	Trend
LOC	185	22	↓
Statements	4	3	↓
Params	1	1	=
Vars	11	6	↓
Cyclomatic Complexity	57	6	↓
Cognitive Complexity	4	7	↑
Nesting Depth	2	3	↑
Return Type Complexity	1	1	=
Code Smells (PMD)	2	0	↓
Deleted	6	169	↑
Churn	12	175	↑
Predizione Modello	BUGGY (TRUE)	BUGGY (HI Risk)	

```
org.apache.openjpa.persistence.jdbc.  
AnnotationPersistenceMappingParser.parseMember  
MappingAnnotations(FieldMetadata)
```

RQ3 What-If Analysis No Smells

La rimozione dei Code Smell porta a una riduzione media della probabilità di bug del **29%**

Apache Bookkeeper

Dataset	Actual Bugs	Predicted Bugs (E)
Dataset A	6790	3835
Dataset B+	2755	2129
Dataset B (Fixed)	-	1951
Dataset C	4035	1706

804 metodi riclassificati da Buggy a Clean
Riduzione della probabilità del **29,18%**
Riduzione del rischio globale del **11,84%**

Apache OpenJPA

Dataset	Actual Bugs	Predicted Bugs (E)
Dataset A	53604	35733
Dataset B+	24752	20491
Dataset B (Fixed)	-	17696
Dataset C	28852	15242

7056 metodi riclassificati da *Buggy* a *Clean*
Riduzione della probabilità del **28,51%**
Riduzione del rischio globale del **13,16%**

Minacce alla validità

- **Validità Interna:**
 - L'algoritmo SZZ è un algoritmo euristico
 - Lo strumento di analisi statica PMD presenta un margine di errore intrinseco (5-10%)
 - I dati e il labeling sono condizionati dalla qualità del tracciamento in Jira.
- **Validità Esterna:**
 - Lo studio si concentra su due progetti dell'ecosistema Apache. I risultati potrebbero variare in contesti industriali differenti.

Lavori Futuri

- **Analisi Longitudinale Post-Refactoring:**
Monitorare l'evoluzione temporale dei metodi rifattorizzati per determinare il "tempo di assestamento" in cui il beneficio strutturale prevale sul rischio iniziale di churn.
- **Integrazione nel processo di Sviluppo (CI/CD):**
Implementare il modello Random Forest nelle pipeline di Continuous Integration per fornire agli sviluppatori feedback in tempo reale sull'impatto delle modifiche strutturali.
- **Estensione della Validità Esterna:**
Replicare lo studio su un corpus eterogeneo di progetti software, estendendo l'analisi a sistemi scritti in linguaggi di programmazione diversi da Java.
- **Labeling Assistito da ML:**
Utilizzare tecniche di *Natural Language Processing* per analizzare semanticamente commit e ticket, migliorando la precisione dell'algoritmo SZZ nell'identificare la versione di iniezione dei bug.



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA